

始めに

[Kaggle Expert になった記録と方法](#)

課題

リンク：[Santa 2025 - Christmas Tree Packing Challenge | Kaggle](#)

この再定義された最適化問題では、サンタがクリスマスツリーの飾りを最小の 2 次元サイズの小包に収められるように手助けしてください。サンタは、1 本から 200 本のツリーを含む発送物を効率よく世界中に郵送するため、最小サイズの正方形の箱の寸法を必要としています。

(評価方法)

提出物は、各パズルにおける正方形のバウンディングボックスの正規化された面積の合計で評価されます。n 本のツリー構成ごとに、ツリーを囲む正方形の一辺 s を二乗し、それをツリーの本数 n で割ります。最終スコアは、すべての構成の合計です。

$$\text{score} = \sum_{n=1}^N \frac{s_n^2}{n}$$

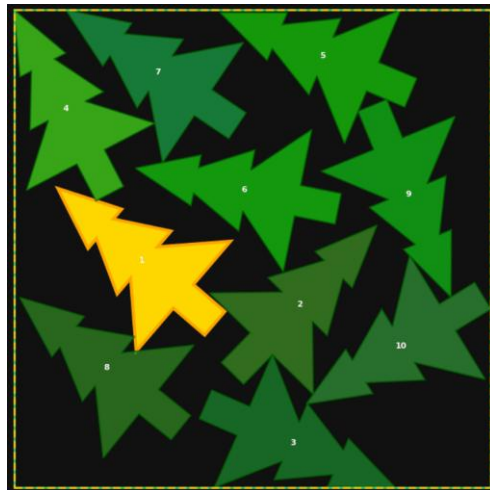
(提出ファイル)

提出ファイルでは、各 id (n 本のツリー構成における 1 本のツリーを表す) に対して、ツリーの位置 (x, y) と回転角度 (deg) を報告する必要があります。ファイルの保存および読み込み時の精度損失を防ぐため、各値は文字列に変換し、先頭に「s」を付けて提出してください。ツリー同士が重なっている提出物はエラーとなります。極端なリーダーボードスコアを避けるため、位置の値には制約が設けられています。(-100 ≤ x, y ≤ 100)

```
id, x, y, deg
001_0, s0.0, s0.0, s20.411299
002_0, s0.0, s0.0, s20.411299
002_1, s-0.541068, s0.259317, s51.66348
etc.
```

イメージ

[Santa 2025 - Christmas Tree Packing Challenge | Kaggle](#)



コード①：Fork of santa-tree-challenge 🤖 🤖

リンク：[Fork of santa-tree-challenge](#) 🤖 🤖

🔧 1. 環境セットアップ

```
INPUT_SUB = '/kaggle/input/submission19/submission.csv'
```

```
INPUT_BIN = '/kaggle/input/santa-submission/bbox3'
```

```
WORKING_DIR = '/kaggle/working/'
```

- 提出ファイルと実行バイナリ (bbox3) を作業ディレクトリにコピー。
- bbox3 は外部最適化ツールで、ランダムなパラメータで配置を改善する目的。
- `os.chmod('./bbox3', 0o755)` で実行権限を付与。

🌲 ツリー形状テンプレートとスコア関数

```
make_polygon_template()
```

- Numba で高速化された関数。
- ツリーの形状 (幹・枝・先端) を構成する頂点座標を返す。

```
score_group(xs, ys, degs, tx, ty)
```

- 各ツリーの回転・位置を反映し、全体のバウンディングボックスを計算。
- 正方形の一辺の長さを二乗し、ツリー数で割った値がスコア。

ChristmasTree クラス

- Decimal を使って高精度で座標・回転角度を扱う。
- scale_factor = 1e18 により、浮動小数点誤差を防ぐ。
- shapely を使ってポリゴンを回転・平行移動し、最終的なツリー形状を生成。

ユーティリティ関数群

load_configuration_from_df(n, df)

- n 本のツリー構成に対応する行を抽出し、ChristmasTree オブジェクトに変換。

get_score(trees, n)

- 全ツリーのポリゴンを結合し、バウンディングボックスの面積を計算。
- n で割ることで正規化。

has_overlap(trees)

- STRtree (空間インデックス) を使って高速に重なりチェック。
- intersects かつ not touches の場合、重なりと判定。

eval_df_sub(df, verb)

- 1~200 本のツリー構成それぞれについてスコアを計算。
- 重なりがある構成は failed に記録。
- 総スコアと失敗リストを返す。

メインループ

while new_score == initial_score:

- スコアが改善されない限り、bbox3 をランダムパラメータで再実行。
- n はツリー数、r はランダムシードや回転に関するパラメータと推測。
- 実行後、再度スコアと重なりチェック。

重なりがあった場合の復元処理

df.loc[mask, ["x", "y", "deg"]] = df_orig.loc[orig_mask, ["x", "y", "deg"]].values

- 重なりが発生した構成のみ、元の提出ファイルから位置・角度を復元。
- submission.csv を上書き保存。

全体の目的

このスクリプトは、提出ファイルのスコアを評価しつつ、外部ツール bbox3 による改善を試み、重なりが発生した場合は安全に元に戻すことで、提出の品質を保ちつつスコア向上を狙うものです。

全体像

提出ファイル (submission.csv) を読み込む → スコアを計算する →
外部ツール bbox3 をランダムパラメータで実行して改善を試す →
もし重なりが出たら元に戻す → 改善されるまで繰り返す

目 ステップごとの流れ

① 必要ファイルを作業ディレクトリにコピー

- 提出ファイル (submission.csv)
- 最適化バイナリ (bbox3)

を /kaggle/working/ にコピーして、bbox3 に実行権限を付けます。

② ツリー形状の定義 (Polygon)

ChristmasTree クラスで、

- ツリーの形状 (ポリゴン)
- 回転角度
- 位置 (x, y)

を Shapely で正確に扱えるように準備します。

※ ここは「評価のための内部表現」を作っているだけ。

③ 提出ファイルを読み込み、各 n=1~200 のスコアを計算

eval_df_sub(df) がやっていること：

1. n=1~200 のグループごとにツリーを読み込む
2. バウンディングボックスの面積 (正方形) を計算
3. n で割ってスコア化
4. 重なりがあるかチェック
5. 全スコアを合計

→ 最初の提出のスコア (initial_score) を得る。

④ 改善ループ開始

```
while new_score == initial_score:
```

スコアが改善されるまで bbox3 を何度も実行。

⑤ bbox3 をランダムパラメータで実行

```
n = random.randint(50, 500) r = random.randint(10, 50) subprocess.run(["./bbox3", "-n", n, "-r", r])
```

- n: どの構成を改善するか (50~500 の中からランダム)
- r: ランダムシードのようなもの

bbox3 が submission.csv を書き換える。

⑥ bbox3 実行後の submission.csv を再評価

- 新しいスコアを計算
- 重なりがあるかチェック
-

⑦ 重なりがあった場合は「その n だけ」元に戻す

```
df.loc[mask, ["x","y","deg"]] = df_orig.loc[orig_mask, ["x","y","deg"]]
```

bbox3 が壊した部分だけ修復して、他の改善は残す

コード②：💡 A Super Simple Improvement Strategy

リンク：💡 [A Super Simple Improvement Strategy](#)

🧠 戦略の本質：n 本の配置は単調非増加であるべき

n 本のツリー配置は、n+1 本の配置をそのまま使って、余った 1 本を置かなければよい。

- n=50 の配置は、n=51, 52, ..., 200 のどれかの配置を流用できる
- だから n=50 のバウンディングボックスのサイズは、n=51~200 のどれよりも小さくあるべき

① スコアが直接改善する

② 新しいレイアウトのヒントが得られる

例えば：

- n=80 の配置が S=1.2
- n=120 の配置が S=1.0

なら、n=80 は 本来もっと詰められるはず ということがわかる。

③ n 本の配置を n+1 本の配置からコピーするだけで改善できる

💡 コードがやっていること

● 第1パス：全 n の配置を読み込み、バウンディングボックスを計算

```
configs[n] = trees box_sizes[n] = get_bounding_box_side(trees)
```

- submission.csv から n 本のツリー配置を読み込む
- Shapely でポリゴンを作る
- 全ツリーの外接正方形の一辺 S を計算
- configs[n] に保存

● 第2パス：単調性チェック → 改善できる n を修正（戦略の核心）

```
for larger_n in range(n + 1, max_n + 1): if box_sizes[larger_n] <= current_box_size:
    best_larger_n = larger_n
```

意味：

- n より大きい本数の配置の中で
- n の配置より小さい S を持つものを探す

見つかったら：

```
better_trees = configs[best_larger_n][:n] configs[n] = better_trees
```

つまり：

n 本の配置を、 $n+1$ 本以上の配置の先頭 n 本に置き換える

● 重なりチェック

```
if has_overlap(trees): failed_overlap_n.append(n)
```

Shapely の STRtree を使って高速に重なりを検出。

● スコア計算

```
current_score = get_score(trees, n) total_score += current_score
```

改善後の配置でスコアを再計算。

Intergration of Existing Result(current best 🚀🚀)

リンク：[Intergration of Existing Result\(current best 🚀🚀\)](#)

🎯 目的

1~40 個以上の公開 submission.csv を全部集めて、それぞれの $n=1\sim200$ の中から“最も良い配置”を選び、最強の 1 つの提出ファイルを作る。

💡 「 $n=1$ の最強解」

💡 「 $n=2$ の最強解」

...

💡 「 $n=200$ の最強解」

を、全ての公開 notebook から自動で抽出して合体する。

✓ 1~200 の各 n が完全に独立しているため、アンサンブルが成立する

✓ 新しい notebook が出たら追加するだけで提出が強化される

✳️ コードの流れをわかりやすく説明

① 大量の submission.csv を working ディレクトリにコピー

```
shutil.copy('/kaggle/input/why-not/submission.csv', '/kaggle/working/submission1.csv')
shutil.copy('/kaggle/input/santa-submission/submission.csv',
'/kaggle/working/submission2.csv')
...
shutil.copy('/kaggle/input/submission14/submission.csv',
'/kaggle/working/submission40.csv')
```

👉 40 個以上の公開 notebook の提出ファイルを全部集める

② submission_files = [...] で全ファイルをリスト化

```
submission_files = [
    '/kaggle/working/submission1.csv',
    '/kaggle/working/submission2.csv', ... '/kaggle/working/submission40.csv', ]
```

👉 このリストを使って「全ファイルを順番に読み込む」

③ ChristmasTree クラスでツリー形状を正確に扱う

- Shapely でポリゴン化
- Decimal で高精度
- 回転・平行移動を正確に再現

👉 どの submission.csv でも 正確に重なりチェック・スコア計算 ができる

④ load_configuration_from_df(n, df)

👉 submission.csv の中から「n 本のツリー配置」だけを取り出す

⑤ get_bounding_box_side() / get_score()

👉 ツリー群の外接正方形の一辺 S を計算

👉 スコア S^2/n を計算

⑥ has_overlap()

👉 Shapely の STRtree で高速に重なりチェック

👉 重なりがある提出は除外できる

⑦ score_and_validate_submission()

ここがアンサンブルの心臓部。

この関数がやること：

✓ Step 1：各 submission.csv を読み込む

✓ Step 2：n=1~200 の配置を全部読み込む

✓ Step 3：各 n の bounding box サイズを記録

✓ Step 4：単調性チェック (n の配置は n+1 以上より悪くてはならない)

- ✓ Step 5: より良い配置があれば置き換える
- ✓ Step 6: 重なりチェック
- ✓ Step 7: スコア計算
- ✓ Step 8: 最強の提出を構築

参考：1st place 解法

[1st place: genetic algorithm and GPU relaxation | Kaggle](#)

ソリューションを紹介するノートブックはこちらです。まずこのノートブックを読むと、この解説の理解が深まると思います：

<https://www.kaggle.com/code/jeroencottaar/santa-2025-1st-place-solution>

200 個すべての解を可視化した別ノートブック：

<https://www.kaggle.com/code/jeroencottaar/santa-2025-1st-place-first-share>

GitHub：

<https://github.com/jcottaar/packing>

🔑 ポイント

1. 階層構造を持つ遺伝的アルゴリズム (GA) が中核

個体同士が階層的に相互作用する GA を中心に据えています。

一つの解が集団全体を支配してしまわないようにしつつ、遺伝情報を交換できるように複数の仕組みを組み込んでいます。

2. GPU を用いた高速な緩和 (relaxation) で局所最適を探索

これを GA と組み合わせることで、より多くの「改善につながる動き」を受け入れられるようになります。GPU 上で最適化を行うことで、局所最適化が非常に高速に進みます。

3. 大きな N に対しては「対称性」が自由度削減の鍵

- ツリー本数が偶数のときは **180° 回転対称**を強制
- 初期解には **結晶構造 (tessellated seeds)** を使用
- 内側は結晶として固定
- 外側のツリーだけを最適化

4. AI を積極的に活用

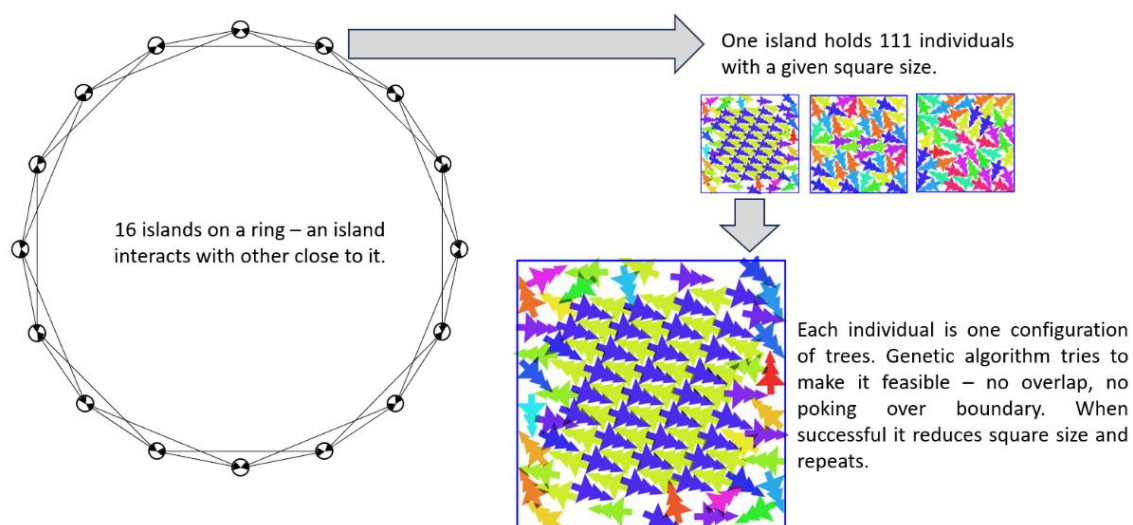
- 文献調査
- アイデアのブレインストーミング
- コードの一部生成
- この解説文も AI による軽い編集を含む

ChatGPT を主に議論用に使い、コード生成には Claude 系モデルも使用したとのことです。

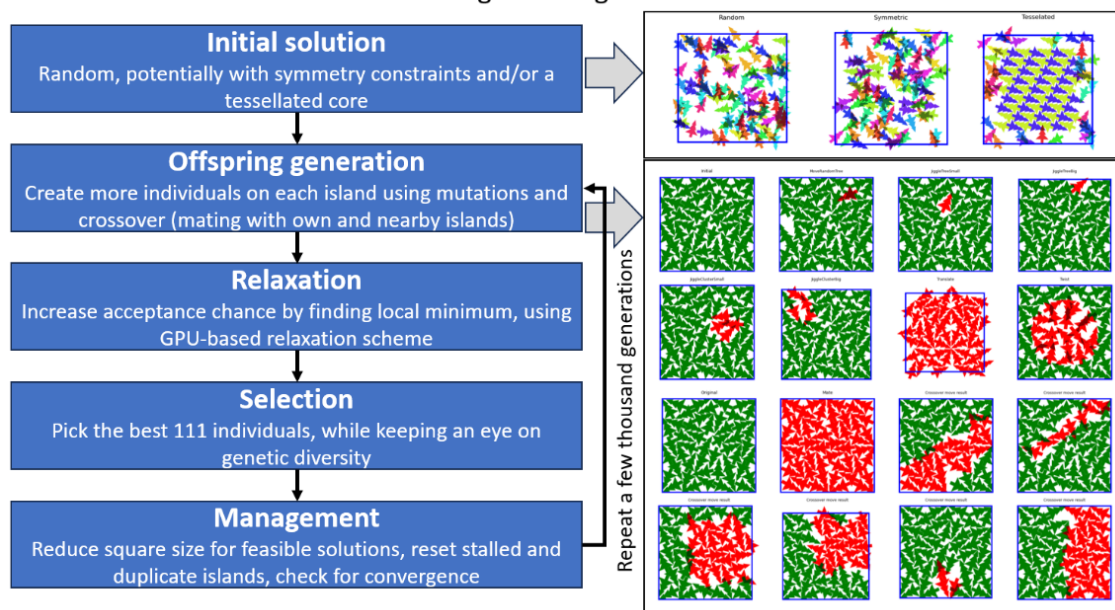
5. 計算資源について

- vast.ai で約 250 ドルを使用
- RTX 5090 を 1000 時間利用
 - 半分は最終解の探索
 - 残り半分はテストと開発
- AI ツールの利用コストは含まず

Hierarchical population for a genetic algorithm



The genetic algorithm



最も基本的な初期解は、ツリーをある程度大きめに設定した正方形の中に、ランダムな位置と角度で散らすというものです。これは N が 40 程度まではうまく機能しますが、それ以上になると自由度を減らすための追加制約が必要になります。

ツリーの本数が偶数の場合、アルゴリズムが 180° 回転対称の解に収束しやすいことに気づきました。それが本当に最適なのか、あるいはアルゴリズムの性質によるものなのかは分かりませんが、ここに明確なチャンスがありました。

つまり：

- ツリーの半分だけをランダムに配置する
- 残り半分は、それらを 180° 回転させてコピーする

という方法です。

90° 対称も試しましたが、見た目は非常に良い解が得られるものの、最適解にはなりませんでした。この 180° 対称を使うことで、 $N \approx 70$ (偶数のみ) まで対応できます。

さらに、多くの解が 結晶構造 (tessellation) を持つことに気づきました。つまり、解の大部分が「格子状のパターン」に従っているのです。

具体的には：

- ほとんどのツリーを あらかじめ決めた結晶 (crystal) から配置する
- 残りのツリーを 外周にランダムに散らす

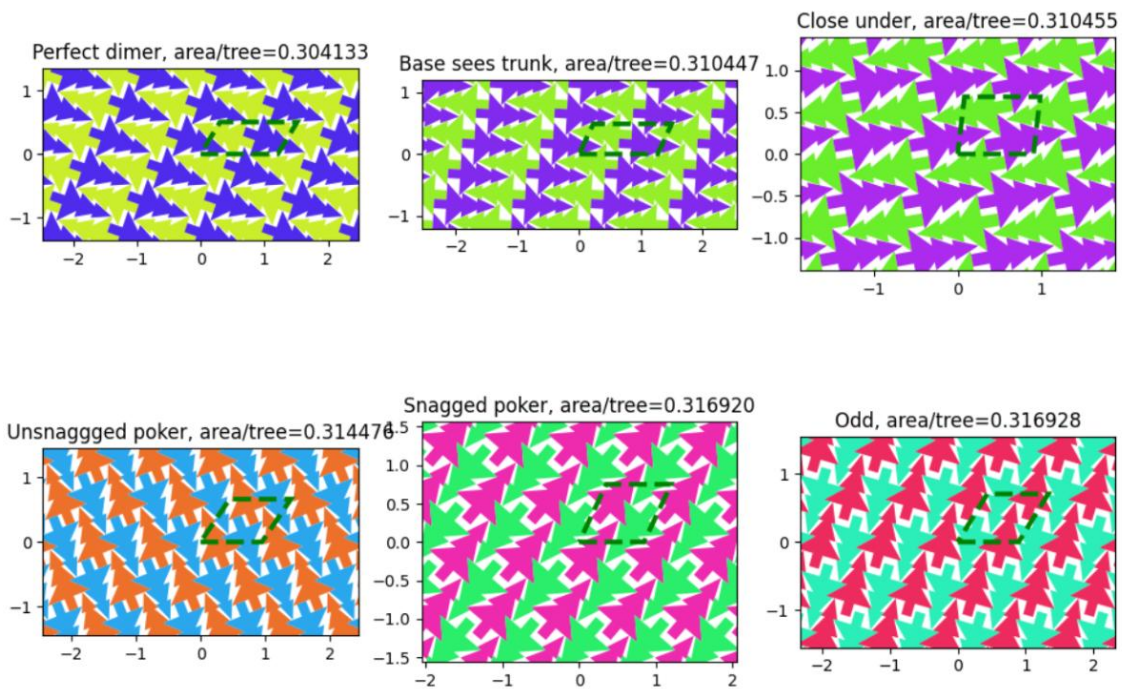
このモードでは、遺伝的アルゴリズムの操作中に 内側のツリーは動かしません。ただし、GPU による局所緩和 (relaxation) ステップでは動かします。テッセレーション (結晶構造) を使うことで、 $N=200$ まで効率的に対応可能になります。

私は主に、偶数 N に対しては 180° 対称を引き続き適用しました。

重要な点として、どの結晶を使うか、その位置や向きをどうするかが挙げられます。

周期境界条件を使った完全な GA を用いて探索した結果、最終的には 2 ツリーの結晶 (dimer) しか見つかりませんでしたが、それでも多様性は十分でした。

以下が私が見つけた結晶の例です (他にも多くありますが、これらの基本形から連続的に変形して到達できます)。



🌱 子個体 (Offspring) の生成

GA の各イテレーションの最初のステップは、

個体数を 3750 まで増やすために子個体を生成すること です。

子個体の生成方法は大きく 2 種類あります：

1. **Mutate** (突然変異)：1 個体に対して何らかの変更を加える
2. **Crossover** (交叉)：2 個体を組み合わせる

🔧 使用される突然変異 (Mutation) 操作

(毎回、以下のうち 1 つだけ を適用)

Move

- ランダムに選んだ 1 本のツリーを
ランダムな位置・角度へ移動する。

Jiggle

- 1 本または複数のツリーに
位置と角度の小さなオフセットを加える。
- この操作には **4 種類**のバリエーションがあり、
- 動かすツリーの数
- 動かす距離
が異なる。

Twist

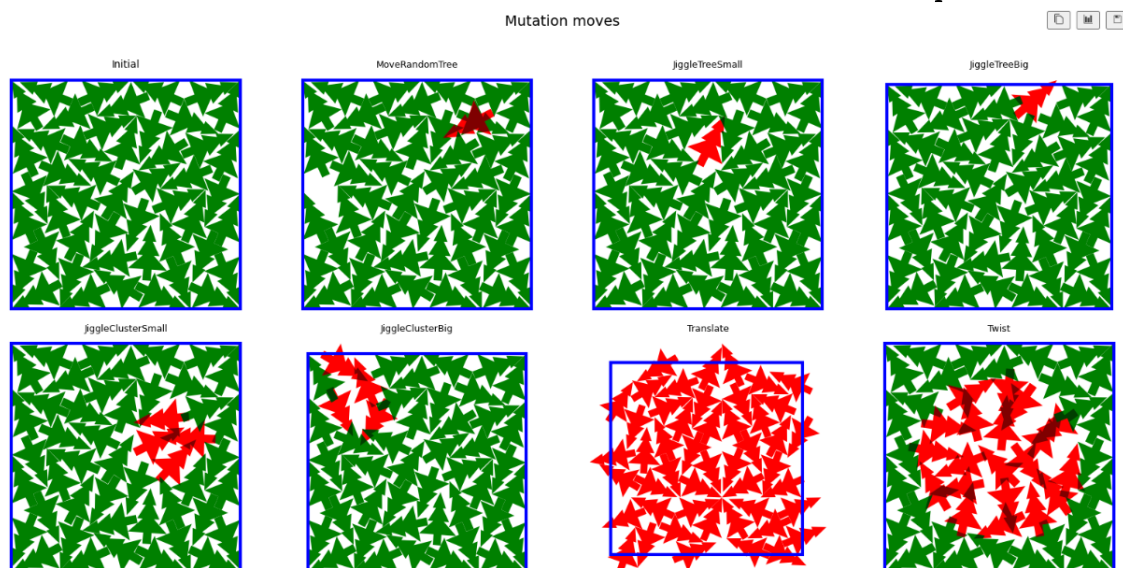
- ある基準点の近くにあるツリーを、

ランダムに選んだ角度だけ回転させる。

- 基準点から遠いツリーほど回転量を小さくすることで、
配置の連続性を保つ。

Translate

- すべてのツリーにランダムな平行移動を加える。
- 正方形の外に出たツリーは 境界をまたいで反対側に回り込む (wrap around)。



交叉 (Crossover) 操作では、

ある解からツリーの一部分を切り取り、

別の個体 (mate) の同じ形状の領域を挿入します。

ここで最も重要なのは「どの個体を mate として選ぶか」です。

mate の候補は次の 2 つのグループから選ばれます。

1. 同じ島 (island) 内のどの個体でも mate として適切

同じ島に属する個体同士は自由に交叉できます。

2. リング上で近くに位置する島の“低スコアのチャンピオン”

リング構造で近接する島の中から、

スコアが低い (=まだ発展途上の) チャンピオン個体を mate として選びます。

これは重要な工夫で、理由は次の通りです：

- 若い島 (まだ良い解が育っていない島) に時間を与えるため
- もし高スコアの成熟した島とすぐ交叉してしまうと、
若い島の個体が一瞬で上書きされてしまう
- それを防ぐために、最初は「低スコアのチャンピオン」だけを許可する

例外：島のスコアがしばらく改善しない場合

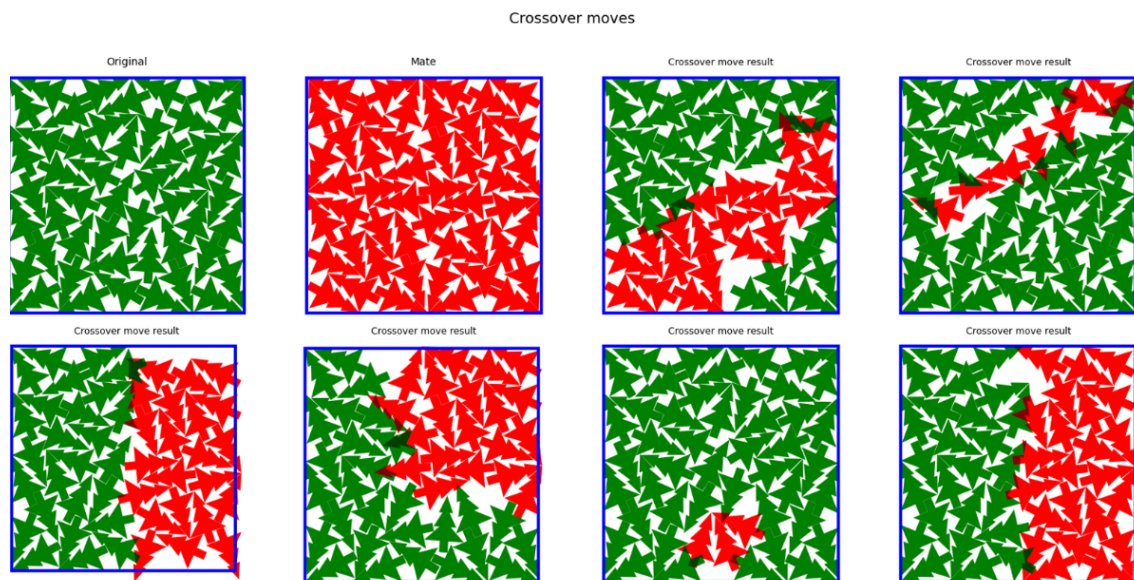
もしある島が長い間スコアを改善できなかった場合、

この制限は解除されます。

その場合：

- 近くの島の どのチャンピオンとも交叉可能
- つまり「成熟した強い個体」も mate として使える

これにより、停滞した島が再び進化できるようになります。



⚡ GPU 緩和 (GPU relaxation)

各アイランドで生成された 3750 個体のうち、

重なりが最も少ない 1500 個体を選びます。

この時点では、ほとんどの解はまだ実用的とは言えません

(つまりツリー同士が大きく重なっています)。

そこで、次のコスト関数の **局所最小値** を探索します。

値が **0** になれば、その解は完全に実行可能 (feasible) です。

🔍 outside と overlap の意味

- **outside**
ツリーが正方形の外にはみ出している量
(完全に内側なら 0)
- **overlap**
ツリー同士の重なり量
(重なっていなければ 0)

overlap の計算には複数の関数を実装していますが、

最も重要なのは **exact separation distance (最小分離距離)** です。

これは：「重なりを解消するために、ツリーをどれだけ動かせばよいか」を表す量です。

しかし、この計算を逐次行うのは非常に重いため、**Minkowski 幾何学**を使って事前計算し、

3 次元（相対 X, Y, 角度）のルックアップテーブルに保存しています。
このテーブルは 三線形補間（trilinear interpolation） で参照されます。

🧠 LBFGS による最適化（CUDA 実装）

コスト関数の最小化には **LBFGS** を使用します。

LBFGS を使うためには：

- コスト関数
- すべてのツリー位置に対する勾配

を計算する必要があります。

これらは **CUDA** 上で**直接実装**されています。作者の LBFGS 実装は PyTorch をベースにしており、**複数の個体（tree sets）**を同時に処理できるようにバッチ対応へ改良されています。

✳️ 選択（Selection）

次に、最良の個体を残すための **選択ステップ** を行います。このステップは **島ごとに個別**に適用されます。（この時点で各島には 1500 個体が存在します。）

ここから **111 個体** に絞り込みます。

まず最初の 27 個体

最初の 27 個体は、**重なり（overlap）**が最も少ない **27 個体** をそのまま選びます。

（上で説明したコスト関数に基づく。）この中には、同一の個体が複数含まれている可能性もあります。

残りの 84 個体

残りの 84 個体は、**重なり**の少なさと**遺伝的多様性（genetic diversity）**の両方を基準に選びます。

具体的には：

- すでに選ばれた個体と比べて
一定以上の**遺伝的多様性**を持つ
- かつ **重なり**が少ない

という条件を満たす個体を選びます。

つまり：

既存の選抜個体と“似すぎている”個体は除外する

ということです。

🧬 遺伝的多様性（genetic diversity）の定義

解 A と解 B の遺伝的多様性とは：

A に対して、どのツリーをどれだけ平行移動・回転させれば

B（または B の鏡像／90° 回転版）と一致するか、

その最小変換量

を指します。

これは ハンガリアンアルゴリズム を使って計算できますが、

このステップでは高速化のために近似を使っています。

ハンガリアン法とその近似の両方が CUDA で実装されています。

✖ 管理 (Management)

最後に、いくつかの管理的な処理を行います。

島のリセット (reset)

以下の 2 つの条件のいずれかを満たす場合、

その島は 新しいランダム初期解で再初期化 (reset) されます。

1. スコアがしばらく改善していない場合

(典型的には約 100 世代)

かつ、その島が現在の最良島ではない場合。

2. その島のチャンピオン (最良個体) が

別の島のチャンピオンと 完全に同一 の場合。

(この判定には、先ほどの遺伝的多様性を

今度は 完全版のハンガリアン法 で計算する。)

正方形サイズの縮小

ある島の個体群の 重なりが一定以下 になった場合、

その島は「現在の正方形サイズで有効解を見つけた」とみなします。

すると：

- その島の 全個体に対して正方形サイズを縮小 します
- 島ごとに異なる正方形サイズで作業することがある
- ただし 同じ島の中では常に同じ正方形サイズ

というルールです。

終了条件

全体の集団が 数世代にわたって改善しなくなった場合、アルゴリズムを停止します。

(これが唯一の停止条件。) 停止時点では、通常まだわずかな重なりが残っているため、最

後に “合法化 (legalize)” ステップ を適用します。これは GPU 緩和に似ていますが、

最終的に完全な重なりゼロの解に到達するよう調整された処理 です。